Ticketing Platform - Database Design

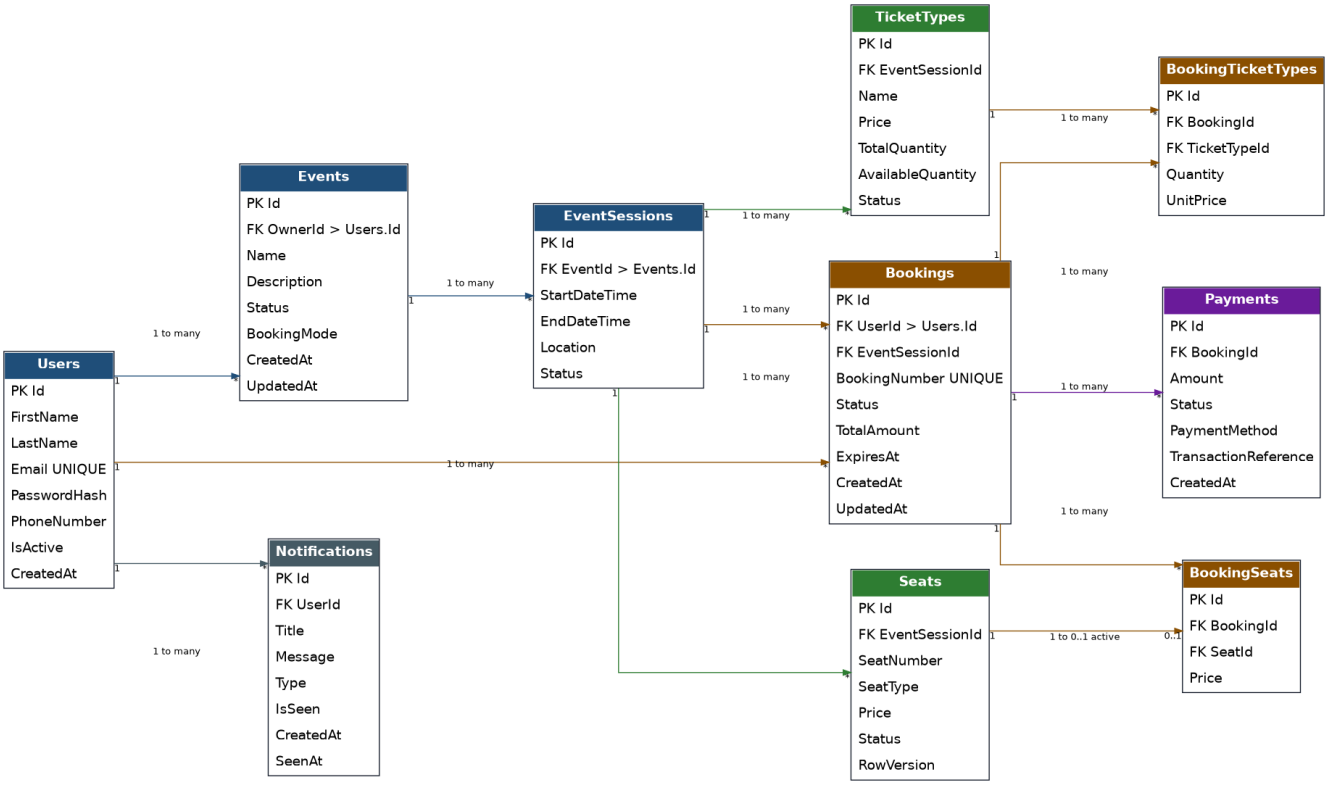
Professional ERD and schema notes for a PostgreSQL-backed ASP.NET Core ticketing system.

1. Overview

This design supports three roles: User, Event Owner, and Admin. Event owners create events and track their own assets; users browse and book; admins monitor the platform.

The system supports two booking modes: seat-based booking and ticket-based booking. The booking flow is modeled like an e-commerce order: Bookings is the main business entity, and booking details are stored in child tables.

2. High-Level ERD



3. Main Tables

Users	Notes
Id PK	Primary key
FirstName, LastName	User profile fields
Email UNIQUE	Used for login and identity lookup
PasswordHash	Use ASP.NET Core Identity where possible
PhoneNumber	Optional or unique depending on business rules
IsActive, CreatedAt	Account status and audit

Events	Notes
Id PK	Primary key
OwnerId FK	References Users.Id; event owner relationship
Name, Description	Event content
Status	Draft, Published, Cancelled, Archived
BookingMode	Seats or Tickets
CreatedAt, UpdatedAt	Audit fields

EventSessions	Notes
Id PK	Primary key
EventId FK	References Events.Id
StartDateTime, EndDateTime	Actual bookable time
Location	Session location
Status	Active, Cancelled, SoldOut, Completed

Seats	Notes
Id PK	Primary key
EventSessionId FK	References EventSessions.Id
SeatNumber	Example A1, A2, VIP-10
SeatType	VIP, Regular, Premium
Price	Current seat price
Status	Available, Reserved, Booked, Disabled
RowVersion	Concurrency token

TicketTypes	Notes
Id PK	Primary key
EventSessionId FK	References EventSessions.Id
Name	VIP, Regular, Early Bird
Price	Current unit price
TotalQuantity, AvailableQuantity	Inventory
Status	Active, Disabled, SoldOut

Bookings	Notes
Id PK	Primary key
UserId FK	References Users.Id
EventSessionId FK	References EventSessions.Id
BookingNumber UNIQUE	Readable reference number
Status	Pending, Confirmed, Cancelled, Expired, Failed
TotalAmount	Final calculated amount

ExpiresAt	Temporary reservation expiry
CreatedAt, UpdatedAt	Audit fields

BookingSeats	Notes
Id PK	Primary key
BookingId FK	References Bookings.Id
SeatId FK	References Seats.Id
Price	Seat price snapshot at booking time

BookingTicketTypes	Notes
Id PK	Primary key
BookingId FK	References Bookings.Id
TicketTypeId FK	References TicketTypes.Id
Quantity	Number of tickets
UnitPrice	Ticket price snapshot at booking time

Payments	Notes
Id PK	Primary key
BookingId FK	References Bookings.Id
Amount	Payment amount
Status	Pending, Paid, Failed, Refunded
PaymentMethod	Card, Wallet, Cash, Gateway
TransactionReference	Gateway reference
CreatedAt	Audit field

Notifications	Notes
Id PK	Primary key
UserId FK	References Users.Id
Title, Message	Notification content
Type	Booking, Payment, Event, System
IsSeen	Read state
CreatedAt, SeenAt	Notification timestamps

4. Constraints and Indexes

```
Users(Email) UNIQUE
Events(OwnerId)
EventSessions(EventId)
Seats(EventSessionId, SeatNumber) UNIQUE
TicketTypes(EventSessionId, Name) UNIQUE
Bookings(BookingNumber) UNIQUE
Bookings(UserId), Bookings(EventSessionId), Bookings(Status)
BookingSeats(BookingId), BookingSeats(SeatId)
BookingTicketTypes(BookingId), BookingTicketTypes(TicketTypeId)
Payments(BookingId), Payments(Status)
Notifications(UserId), Notifications(IsSeen)
```

5. Critical Business Rules

Booking mode: If Event.BookingMode = Seats, use BookingSeats only. If Event.BookingMode = Tickets, use BookingTicketTypes only.

Session matching: Seat.EventSessionId or TicketType.EventSessionId must match Booking.EventSessionId.

Price snapshot: Booking detail tables store the price at booking time to protect historical data.

Seat concurrency: A seat cannot be linked to more than one active booking. Use transactions and concurrency control.

Payment attempts: One booking can have multiple payment attempts, for example one failed attempt followed by a successful payment.

6. PostgreSQL Implementation Notes

Use database transactions for booking creation. PostgreSQL should remain the source of truth. Redis can be added later for faster temporary availability checks and real-time seat reservation caching.

For ASP.NET Core Identity, the implementation may use Identity-generated tables such as AspNetUsers, AspNetRoles, and AspNetUserRoles. The Users table in this document represents the application-level user concept.